

ExpenseSense: Privacy-Preserving On-Device Personal Finance Tool Calling with Sub-2B LLMs

Anonymous

Abstract

Personal finance data—spending habits, health-related purchases, temporal patterns—is among the most sensitive information people generate daily. Yet natural-language financial assistants today require transmitting these queries to cloud-hosted LLMs, exposing users to inference attacks, data retention, and third-party data flows that violate the contextual norms under which that data was created. We introduce **ExpenseSense**, a privacy-first personal finance assistant that runs entirely on-device, ensuring that no financial records, query text, or model outputs leave the user’s device. To evaluate whether this privacy architecture is practically viable, we benchmark on-device tool calling using a 115-question test suite across six sub-2B quantized SLMs (0.8B–2B), comparing single-agent (joint routing and parameter extraction) against dual-agent (router-specialist) pipelines. Architectural decomposition substantially improves both accuracy and safety: dual-agent Qwen3.5-2B achieves 77.6% strict Correct Ratio (+17.4 pp over single-agent) while reducing average latency by 48%, and dual-agent Qwen3.5-0.8B reaches 89.1% Task Accuracy in 3.0 s with 1.2 GB RAM. Critically, dual-agent architectures shift error modes toward safer failures—missing rather than spurious parameters—a property that directly reduces the risk of silent incorrect financial output reaching users. Our results demonstrate that private, high-quality financial analytics is achievable on consumer hardware today, with no cloud dependency. All code, data, and prompts are available on GitHub.

1 Introduction

Large language models (LLMs) have rapidly advanced tool-augmented AI (??), enabling agents to invoke structured APIs from natural language. However, research primarily targets large cloud-hosted models, which is problematic for personal finance where local inference is crucial for privacy.

Expense records reveal sensitive behavioral patterns, yet users expect natural language interfaces (e.g., asking “how much did I spend on food last month?”) that map to structured queries over local data (?). This creates an underexplored problem: can *sub-2B quantized models* running entirely on consumer hardware reliably call domain-specific financial tools from informal, colloquial language, and in doing so, serve as practical privacy infrastructure for real users?

Existing benchmarks (????) focus on models above 7B parameters, which are prohibitively slow for local deployment. Conversely, on-device models like Octopus v2 (?) and Hammer (?) demonstrate the potential of smaller models but have not been evaluated on domain-specific financial schemas or under consumer-hardware latency constraints.

We address three research questions (**RQ**):

- **RQ1:** Does on-device inference with sub-2B SLMs provide a practically viable privacy protection for personal finance tool use—i.e., can users obtain accurate financial analytics without surrendering query data to cloud providers?
- **RQ2:** Does decomposing intent classification from parameter extraction (dual-agent) yield significant accuracy gains over a joint single-agent architecture?
- **RQ3:** What are the primary failure modes across architectures, and do they differ in the human-facing harms they produce?

This paper introduces **ExpenseSense**, with the following contributions:

1. Empirical **on-device deployment guidelines for privacy**: Qwen3.5-0.8B in dual mode achieves 89.1% Task Accuracy with 3.0 s latency and ≈ 1.2 GB RAM, demonstrating that private, high-quality financial analytics can

run locally on consumer hardware—making on-device inference a practical privacy protection, not merely a theoretical one.

2. A **fine-grained error taxonomy** of 7 categories showing that dual-agent architectures consolidate errors into predictable, safer failure modes—shifting from spurious parameter additions (which silently corrupt query results and can mislead financial decisions) toward missing parameters (which are visible omissions). This error-mode shift is the central human-safety argument for architectural decomposition.
3. A comparative evaluation of **single-agent vs. dual-agent** architectures across six quantized SLMs (0.8B–2.0B parameters), showing that dual-agent decomposition reduces latency (by up to 48% for Qwen3.5-2B), improves exact-match correctness (+17.4 pp Correct Ratio), and stabilizes sub-1B models without cloud resources.
4. A **115-question dataset** for financial tool calling across five analytical tools, with a 7-feature complexity scoring function categorising queries into three difficulty tiers (L1–L3) that simulate real-world user interactions.

2 On-Device Inference as Privacy Protection for Real Users

The deployment of LLMs in personal finance exposes a structural tension: the same conversational interface that makes financial analytics accessible to users also makes their most sensitive behavioral data accessible to cloud providers. When a user types “how much did I spend on therapy last month?” into a cloud-hosted assistant, they are not merely querying a database—they transmit evidence of a health condition, a temporal pattern, and financial commitment to a third-party infrastructure they did not choose and cannot audit. The query is the disclosure.

Q demonstrated that LLMs can infer sensitive personal attributes—including location, income, and demographic characteristics—from seemingly innocuous text, and Q catalog privacy risks including data leakage, memorization, and inference attacks. The implication for financial assistants is direct: even if the database never leaves the device, routing natural-language queries through

cloud LLMs leaks behavioral signals that can be retained, aggregated, and exploited beyond the user’s control or knowledge.

ExpenseSense addresses this threat by adopting an *on-device, privacy-first* architecture. By leveraging sub-2B quantized models that run efficiently on consumer hardware (tested on an Apple M1 with 8 GB unified memory), all user data, query text, and model outputs remain strictly local. No financial records, queries or inferences leave the device.

This local-first design instantiates Q’s framework of *contextual integrity*: financial data generated for personal expense tracking should remain within that context rather than flowing to third-party providers operating under different informational norms (Q). Routing a query like “did I overspend on food this month?” through a cloud LLM provider violates contextual integrity even when the underlying expense database is local—the query, not just the data, carries private information.

The practical threat model we address covers three distinct risks for users: (1) cloud provider data retention and secondary use of query logs; (2) query-level inference attacks that reconstruct user attributes from financial language patterns (Q); and (3) third-party exposure by provider breaches or subpoenas. On-device inference eliminates all three by ensuring queries never reach external infrastructure.

The key challenge is that on-device inference with sub-2B models has historically been unreliable for structured tool calling (Q). Our work demonstrates that architectural decomposition—separating intent classification from parameter extraction—bridges this reliability gap, making private on-device inference not merely possible but practical.

3 Related Work

Tool-Augmented Language Models. Toolformer (Q) demonstrated self-supervised tool-use learning; Gorilla (Q) and ToolLLM (Q) scaled to thousands of real-world APIs. HuggingGPT (Q) and ToolAlpaca (Q) explored multi-step orchestration. APIGen (Q) introduced a verified pipeline for generating function-calling training data, showing that 1B-parameter models fine-tuned on high-quality synthetic data can surpass GPT-3.5-Turbo on structured tool invocation. These approaches rely on cloud-hosted models ($\geq 7B$), leaving the privacy-constrained on-device setting

largely unaddressed.

Function-Calling Evaluation. The Berkeley Function Calling Leaderboard (?) established a benchmark using AST substring matching and execution-based validation. ? proposed an evaluation suite (FSP, ACS, AVC, Task Accuracy, and Correct Ratio) to evaluate small model function calling without code execution; we adopt these metrics directly. CallNavi (?) proposed a two-stage router+specialist approach—similar to our dual-agent design—but evaluates models $\geq 7\text{B}$ on general-purpose APIs. In contrast, we focus on sub-2B models for domain-specific, single-turn financial tasks under strict privacy constraints.

On-Device Function Calling. MobileLLM (?), Phi-3 (?), and related surveys (?) have explored sub-4B parameter models for mobile deployment. Octopus v2 (?) fine-tuned a 2B model for on-device API calling via functional tokens, achieving competitive latency against GPT-4 on specific tasks. Hammer (?) introduced function masking to reduce overfitting to function names, addressing an issue similar to the category normalisation errors we observe. Apple Intelligence (?) validated on-device tool calling at scale with a $\sim 3\text{B}$ model, demonstrating the viability of the paradigm. ? found that structured output compliance is the primary bottleneck for SLMs, which motivates our dual-agent architecture.

Financial NLP and Privacy. FinQA (?) and FinGPT (?) target financial numerical reasoning and generation, but focus on text generation rather than tool calling under privacy constraints. FinMCP-Bench (?) and FinTrace (?) evaluate LLM agents for financial tool use, but target cloud-scale models. Personal finance assistants (?) require strict privacy, making local execution crucial.

Human-Centered Privacy for Language Models. A growing body of work examines privacy not as a technical property of models but as a human-centered concern shaped by user expectations and social context. ?’s contextual integrity framework has been applied to conversational AI to show that data flows violate privacy when they cross contextual norms, even when no explicit PII is transmitted. ? demonstrated that LLMs act as inference engines, reconstructing sensitive personal attributes from ordinary text—a threat particularly acute when financial queries are routed through cloud providers.

Tool	ID	Key Params
plot_time_series	1	category, year/month, ranges, months, ignore_rent
plot_distribution	2	category, year/month/day, ranges, months, ignore_rent
plot_comparison_bars	3	category, y1/m1, y2/m2, sm/em/ey, ignore_rent
calculate_total	4	category, remarks, year/month/day, months, ignore_rent
get_top_expenses	5	n, category, year/month/day, months, min_amount, ignore_rent

Table 1: Tool inventory. The five tools span visualisation (1–3), aggregation (4), and retrieval (5).

? surveys the resulting landscape of data leakage, memorization, and inference risks in deployed LLM systems. Our work operationalises these concerns at the system level: rather than studying how cloud LLMs violate privacy, we build and evaluate an architecture that eliminates the exposure pathway entirely, keeping all query data local to the user’s device.

4 System Design

4.1 Tool Inventory

ExpenseSense exposes five analytical tools over a user’s expense dataframe (columns: date, amount, major category, subcategory, remarks). Table 1 summarises each tool and its parameter space. These tools span core personal finance use cases: time series analysis, proportional breakdown, period-over-period comparison, sum aggregation, and ranked retrieval.

4.2 Architectural Configurations

We evaluate two local architectures for mapping natural language user queries to tool calls.

Single-Agent Pipeline. The single-agent pipeline uses a single prompt with all five tool definitions, parameter schemas, and joint few-shot examples. The model must select the tool and extract parameters in a single pass, outputting a JSON object containing the tool ID (1–5) and arguments. The prompt includes structured output rules, date-handling instructions, and 7 tool-specific examples aggregated across all 5 tools. Prompt length: $\approx 5,500$ tokens (context window inclusive).

Dual-Agent Pipeline. The task is decomposed into two sequential inference calls:

1. **Router:** A focused intent-classification prompt with 19 few-shot examples and explicit disambiguation rules (e.g., “compare” requires two distinct time periods) elicits a single digit (1–5).
2. **Specialist:** The predicted tool ID selects a tool-specific prompt containing 12–15 examples for that *one* tool, along with detailed parameter definitions and date-handling rules.

The router sees $\approx 1,000$ tokens; the specialist sees $\approx 1,500$ – $2,000$ tokens with deeper, more directly relevant supervision. This simplifies the task for each model call and provides more relevant in-context examples, though it requires two sequential inference passes. Combined prompt tokens: $\approx 2,700$ per query. Both architectures inject a dynamically generated category vocabulary—11 major categories and ≈ 60 subcategory aliases drawn from the user’s own dataset (Appendix B)—into a `{metadata}` block within the prompt, grounding category parameter extraction in the user’s actual expense taxonomy. These category names are inherently sensitive: they reveal what a user spends money on (e.g., healthcare, dining, donations), making their transmission to third-party cloud providers a privacy risk. In dual-agent mode, only the specialist receives this vocabulary; the router operates on pure intent classification without category context, further limiting the surface over which sensitive data is processed.

4.3 Post-Hoc Parameter Validation

A post-processing validation layer corrects category names deterministically without additional model calls:

1. **Exact match** against the known vocabulary.
2. **Morphological normalisation:** `groceries`→`grocery`, `cafés`→`cafe`.
3. **Edit-distance matching** via `diffib` (cutoff ≥ 0.80).
4. **Token overlap scoring** (Jaccard; cutoff ≥ 0.50).

All results report both raw (pre-validation) and validated (post-validation) metrics.

Task Group	Cases	Avg Params	Avg C
Time Series	23	2.9	7.0
Distribution	21	2.5	6.4
Comparison	28	4.4	8.8
Calculate Total	22	2.4	6.3
Top Expenses	21	3.2	6.7
Total	115	3.2	7.1

Table 2: Test case distribution by group. Comparison queries are the most parameter-dense and complex.

5 Benchmark Design

5.1 Test Suite

The benchmark contains 115 test cases across five task groups (Table 2). Queries are written in naturalistic, informal language: abbreviations (“*compare electricity '24 vs '25*”), colloquialisms (“*can ya tell me*”), misspellings (“*distribuuition*”), and domain-specific terms (“*combinis*”, “*nomikais*”, “*shinkansen*”). This mimics the Japan-based personal finance context of the target application and tests robustness to spelling variations, colloquialisms, and cultural terms.

5.2 Complexity Scoring

Each test case is assigned a difficulty score:

$$C = n_p + d_{\text{date}} + d_{\text{cat}} + d_{\text{rel}} + d_{\text{multi}} + d_{\text{abbr}} + d_{\text{hol}} \quad (1)$$

where n_p = number of expected parameters; $d_{\text{date}} \in \{0-4\}$ scores date specification complexity; $d_{\text{cat}} \in \{0-2\}$ is the category normalisation distance; $d_{\text{rel}} \in \{0, 1\}$ flags relative time expressions; $d_{\text{multi}} \in \{0-2\}$ counts compound parameter groups; $d_{\text{abbr}} \in \{0, 1\}$ flags abbreviated dates; and $d_{\text{hol}} \in \{0, 2\}$ flags calendar knowledge requirements. Scores are binned into three tiers: **L1** ($C \leq 4$, 24 cases), **L2** ($5 \leq C \leq 7$, 45 cases), **L3** ($C \geq 8$, 46 cases).

5.3 Evaluation Metrics

We adopt the metric suite of ?:

- **FSP** (Function Selection Precision): Binary indicator of correct tool prediction.
- **ACS** (Argument Completeness Score): F_1 on argument *name* overlap.
- **AVC** (Argument Value Correctness): Fraction of correctly named arguments whose values also match.

Model	Family	Params	RAM
EXAONE-4.0-1.2B	EXAONE-4.0 (?)	1.2B	2.0 GB
Gemma-3-1B-it	Gemma-3 (?)	1B	1.6 GB
LFM2.5-1.2B	LFM2.5 (?)	1.2B	1.4 GB
MiniCPM5-1B	MiniCPM (?)	1B	1.5 GB
Qwen3.5-0.8B	Qwen3.5 (?)	0.8B	1.3 GB
Qwen3.5-2B	Qwen3.5 (?)	2B	2.1 GB

Table 3: Models evaluated. All use Q8_0 GGUF quantization via llama.cpp. RAM is average peak RSS.

- **Task Accuracy:** F_1 on the combined set $\{\text{tool}\} \cup \{k=v\}$, jointly capturing tool selection and parameter correctness.
- **Correct Ratio (CR):** Strict binary indicator—1.0 only when Task Acc = 1.0, i.e. the entire function call is perfectly correct.

All metrics are averaged across 5 repetitions per test case (temperature = 0.1) with 95% bootstrap confidence intervals (?) ($n = 2000$ resamples).

5.4 Error Taxonomy

Each prediction is classified into one of seven mutually exclusive error types: CORRECT, TOOL_WRONG, PARSE_FAILURE, PARAM_MISSING, PARAM_VALUE_WRONG, PARAM_EXTRA, FORMAT_VIOLATION. This taxonomy helps diagnose specific failures over just aggregate accuracy, which is crucial for evaluating the safety profile of on-device financial assistants with respect to real-user harm.

6 Experimental Setup

6.1 Models

All six models (Table 3) use Q8_0 GGUF quantization and run via llama.cpp (?) with full GPU offload on Apple Metal.

6.2 Hardware and Inference Protocol

Experiments run on an Apple M1 (8-core CPU, 8-core GPU, 8 GB unified memory)—a consumer laptop representative of privacy-constrained deployment environments. Parameters: temperature = 0.1, min_p = 0.05, n_ctx = 4096, stop tokens `["\n\n", ""]`. Each model is warmed up through all six prompt templates (one router + five specialists) before evaluation begins. Each of the 115 test cases is run 5 times per model per architecture, yielding $115 \times 5 \times 6 \times 2 = 6,900$ total observations.

7 Results

7.1 Overall Performance

Table 4 presents the full results. Figure 1 shows the accuracy–latency Pareto frontier and the error taxonomy breakdown.

Key findings (RQ1, RQ2). (1) Sub-2B SLMs can reliably call financial tools on consumer hardware (RQ1), with the Qwen3.5 family achieving >89% Task Accuracy in dual mode. This demonstrates that cloud reliance can be avoided while keeping all data—queries and results—private to the user’s device. (2) While optimized single-agent prompts allow Qwen3.5-2B to reach 92.0% Task Accuracy, dual-agent decomposition is still preferred for two reasons (RQ2): (a) Strict correctness—Correct Ratio improves from 60.2% to 77.6% (+17.4 pp) for Qwen3.5-2B, representing a significant reduction in silent parameter errors that could produce incorrect financial analytics. (b) Enabling smaller models—Qwen3.5-0.8B improves from 79.8% to 89.1% Task Accuracy (+9.3 pp) while halving latency (5.8 s to 3.0 s) in dual mode, making sub-1B models viable on low-memory devices.

7.2 Per-Tool-Group Analysis

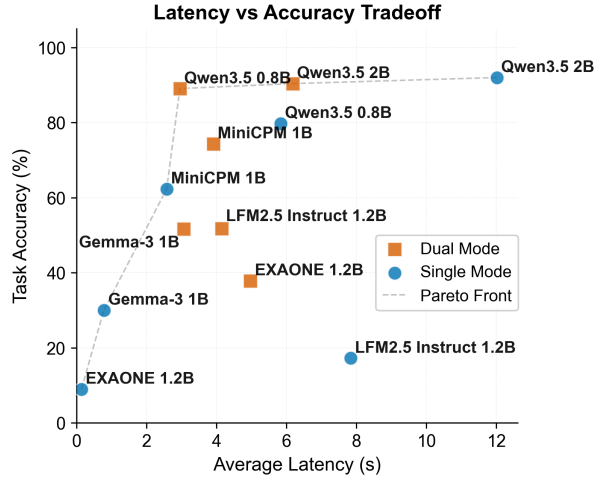
Table 5 shows per-group accuracy for the top two dual-mode models. Figure 2 shows the full model $\times$ category heatmap. `get_top_expenses` reaches 98.9% accuracy with Qwen3.5-2B due to its simple parameter structure. Conversely, `plot_time_series` is the most difficult task (74.4% for Qwen3.5-2B) due to complex date specifications and relative time expressions (e.g., “past X months”) requiring temporal reasoning.

7.3 Complexity Tier Analysis

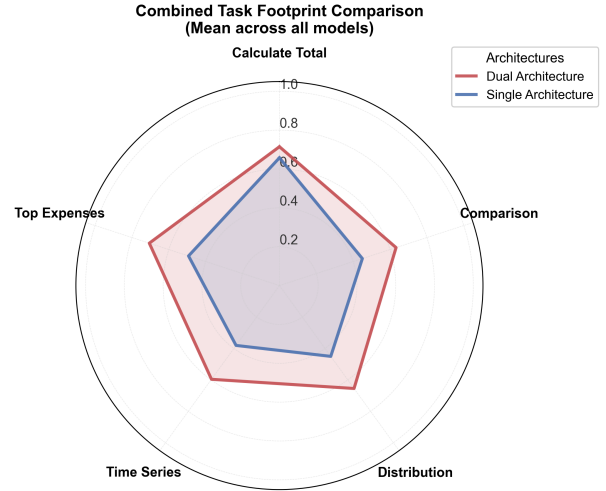
Figure 3 presents the per-model accuracy breakdown across complexity tiers in dual mode. Task Accuracy decreases from 96% (L1) to 82% (L3) for Qwen3.5-2B, and from 95% to 84% for Qwen3.5-0.8B. L3 queries involve constructs like cross-year range comparisons (“*compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026*”), which require extracting up to 10 parameters with complex temporal constraints.

7.4 Error Taxonomy Analysis

Figure 4 visualises the full error distribution. The error distribution shifts significantly between configurations (RQ3).



(a) Accuracy–latency Pareto frontier. The dual-mode Qwen3.5-0.8B (orange square, ≈ 3 s, 89%) sits on the efficient frontier, offering the best accuracy per unit latency. Single-mode Qwen3.5-2B reaches the highest Task Accuracy (92%) but at $4\times$ the latency (12 s).



(b) Task footprint (mean Task Accuracy across all models). The dual architecture (red) uniformly expands coverage over single (blue) on all five task groups, with the largest gains on time series and comparison tools.

Figure 1: System-level results. (a) Dual-agent Qwen3.5-0.8B dominates the efficiency frontier for privacy-preserving deployments. (b) Dual-agent decomposition improves coverage across all task categories.

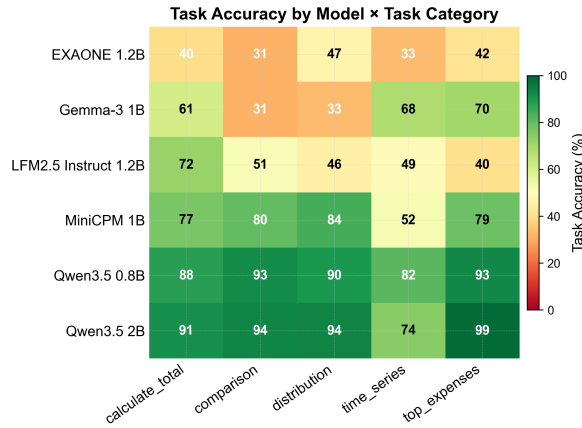


Figure 2: Task Accuracy (%) by Model $\times$ Tool Category in dual mode. Qwen3.5 models achieve $>74\%$ across all categories, while weaker models show pronounced gaps in comparison and time series tasks.

For Qwen3.5-2B: single-agent produces 22.6% `PARAM_EXTRA` and 7.3% `PARAM_MISSING`; dual-agent reduces `PARAM_EXTRA` to 1.9% while `PARAM_MISSING` remains at 8.0%.

The human-facing implications of this shift are significant. A `PARAM_EXTRA` error is a silent corruption: when a model adds a spurious `months=1` filter to the query “how much did I spend on food?”, the system returns an answer—last month’s food spending—that looks correct but answers a different question. A user relying on this figure for tax preparation, expense reporting, or budgeting receives a plausible but wrong number, with no indi-

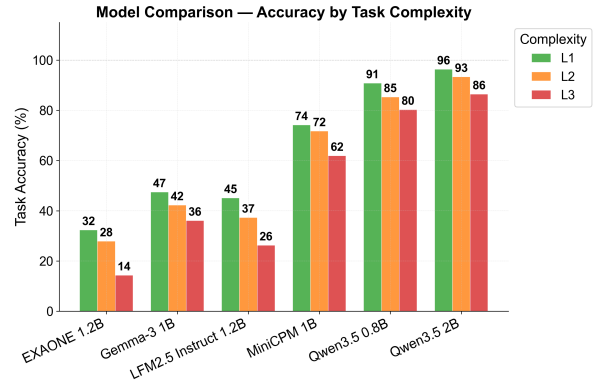


Figure 3: Task Accuracy by complexity tier (L1/L2/L3) per model in dual mode. The Qwen3.5 family degrades gracefully with increasing complexity.

cation that the scope was silently restricted. In contrast, a `PARAM_MISSING` error produces a visibly broader result (e.g., all-time food spending when a month was intended), which is more likely to trigger user scrutiny. The dual-agent architecture’s shift from extra-parameter to missing-parameter failures therefore represents a qualitative improvement in user-facing safety, not just a numerical accuracy improvement.

For Qwen3.5-0.8B: single-agent produces 32.5% `PARAM_MISSING` and 11.8% `PARAM_EXTRA`; dual-agent reduces these to 12.9% and 11.1% respectively, shifting the model from a high-variance to a more predictable failure profile.

Mode	Model	FSP	ACS	AVC	Task Acc	CR	Latency (s)	RAM (GB)
Single	EXAONE-4.0-1.2B	19.3	—	0.3	8.9	0.3	0.1	2.0
	Gemma-3-1B-it	27.0	—	47.5	30.0	0.9	0.8	1.6
	LFM2.5-1.2B	28.7	—	11.7	17.2	3.1	7.8	1.4
	MiniCPM5-1B	71.1	—	86.7	62.3	13.0	2.6	1.5
	Qwen3.5-0.8B	95.0	—	91.2	79.8	40.2	5.8	1.3
	Qwen3.5-2B	96.5	—	97.6	92.0	60.2	12.0	2.1
Dual	EXAONE-4.0-1.2B	90.6	3.7	3.3	37.8	3.0	5.0	1.9
	Gemma-3-1B-it	55.1	59.9	71.6	51.7	16.5	3.1	1.5
	LFM2.5-1.2B	69.6	51.6	57.2	51.8	16.3	4.1	1.4
	MiniCPM5-1B	84.9	77.0	81.6	74.3	43.5	3.9	1.5
	Qwen3.5-0.8B	94.3	90.2	96.8	89.1	61.7	3.0	1.2
	Qwen3.5-2B	92.0	91.6	97.4	90.4	77.6	6.2	2.1

Table 4: Main results (validated metrics, %). **Bold** = best per metric. FSP = Function Selection Precision; ACS = Argument Completeness Score; AVC = Argument Value Correctness; Task Acc = F_1 -based task accuracy; CR = strict exact-match Correct Ratio. ACS is omitted (—) for single-agent configurations where the joint output format makes isolated argument coverage less meaningful. Latency and RAM are per-query averages.

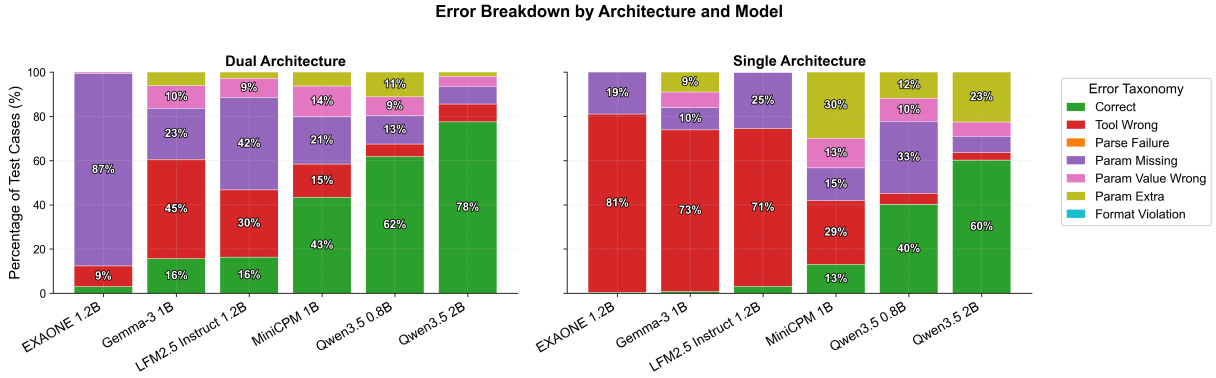


Figure 4: Error taxonomy breakdown (% of test cases) per model and architecture. Single-agent Qwen3.5-2B (right panel) produces 22.6% `PARAM_EXTRA` errors—spurious parameters that could lead to incorrect financial calculations. Dual-agent (left) eliminates most of these, consolidating residual errors into the more predictable `PARAM_MISSING` category.

Group	Qwen3.5-0.8B		Qwen3.5-2B	
	T.Acc	CR	T.Acc	CR
Time Series	81.6	54.8	74.4	60.9
Distribution	89.6	56.2	93.8	79.0
Comparison	92.5	71.4	93.7	73.6
Calc. Total	87.9	59.1	91.4	84.5
Top Expenses	93.3	64.8	98.9	92.4

Table 5: Per-group results in dual mode (validated, %). T.Acc = Task Accuracy, CR = Correct Ratio.

Model (Dual)	Δ Task Acc	Δ CR
EXAONE-4.0	+0.00	+0.00
Gemma-3-1B-it	+0.12	+0.00
LFM2.5-1.2B	+0.50	+0.00
MiniCPM5-1B	+1.17	+0.87
Qwen3.5-0.8B	+0.41	+1.57
Qwen3.5-2B	+0.29	+0.87

Table 6: Validation layer impact (Δ in percentage points, dual mode). Gains are modest but consistent for capable models.

7.5 Token Efficiency

Figure 5 shows that despite using $\approx 2,700$ prompt tokens per query (vs. $\approx 5,500$ for single-agent), the dual-agent architecture achieves equivalent or superior accuracy. The single-agent pipeline processes larger contexts but exhibits lower performance per token since the model must simultaneously attend to all five tool schemas and disambiguation instructions. The dual-agent architecture decomposes the

context, focusing each pass on a single subtask to achieve higher accuracy-per-token.

7.6 Validation Layer Impact

The validation layer (Table 6) contributes modest gains, primarily correcting category casing and normalisation. This reveals that argument extraction rather than string normalisation remains the primary bottleneck, since validation cannot recover

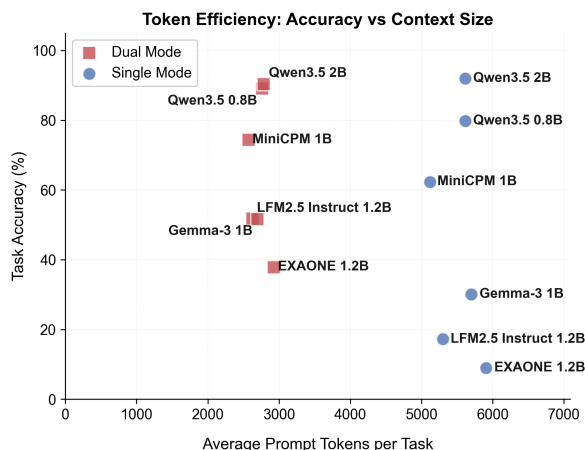


Figure 5: Token efficiency: Task Accuracy vs. average prompt tokens per query. Dual-agent configurations (red) achieve higher accuracy with fewer total tokens than their single-agent counterparts (blue), demonstrating superior context utilisation.

missing parameters (PARAM_MISSING).

8 Discussion

Privacy through Task Decomposition. The dual-agent architecture both improves accuracy and makes local inference viable. By separating routing (single-digit output) from extraction (focused JSON), the dual-agent setup reduces the generation complexity per call, addressing the primary bottleneck of small models. Decomposing the task helps sub-2B models perform reliably on domain-specific schemas—and by doing so, makes on-device inference a practical privacy guarantee rather than an aspirational one.

Why dual-agent still matters despite strong single-agent prompts. Although prompt optimisation increases single-agent performance, dual-agent configurations remain preferable for several reasons: **(1)** The 17.4 pp gap in Correct Ratio (60.2%→77.6%) means that single-agent produces wrong parameters in 40% of queries vs. 22% for dual-agent. **(2)** For the privacy-optimal Qwen3.5-0.8B (which fits in <1.5 GB RAM), dual-agent provides a +9.3 pp Task Accuracy boost *and* halves latency, because shorter, more focused prompts generate faster on small models. **(3)** The error taxonomy shift from PARAM_EXTRA (dangerous in finance) to PARAM_MISSING (safer) represents a qualitative improvement in user-facing safety that aggregate accuracy numbers do not capture.

Analyzing the EXAONE Failure. EXAONE-4.0-1.2B achieves 90.6% FSP in dual mode—excellent routing—but averages near-empty specialist outputs (ACS = 3.7%). This likely points to formatting or parsing issues with the quantized model’s template rather than a capacity limit. Fine-tuning techniques, such as function masking or functional tokens, could help resolve these formatting failures.

Addressing Highly Complex Queries. Even the best model achieves only 59.6% strict Correct Ratio on L3 queries in dual mode, which involve cross-year range comparisons requiring up to 10 interdependent parameters. Fine-tuning on domain-specific examples using verified synthetic data (?) is a potential avenue to improve performance on complex queries.

Deployment Recommendations. Based on our evaluation, we recommend: **(1) Memory-constrained** (≤ 2 GB available): Qwen3.5-0.8B dual, 89.1% Task Acc, 3.0 s, 1.2 GB RAM. **(2) Accuracy-first:** Qwen3.5-2B dual, 90.4% Task Acc, 77.6% CR, 6.2 s, 2.1 GB RAM. **(3) Budget hardware:** MiniCPM5-1B dual, 74.3% Task Acc, 43.5% CR, 3.9 s, 1.5 GB RAM. All configurations keep user data entirely on-device.

Case Study. Figure 6 shows a usage session of ExpenseSense. A user queried the system in informal language—“can ya plot spending on snacks for past 14 months?”—which the dual-agent Qwen3.5-0.8B pipeline correctly routed to `plot_time_series` and extracted the parameters {category: snacks, months: 14}. The resulting visualization revealed a clear declining trend in weekly snack expenditure (from peaks of ¥3,000–4,000 to under ¥1,000), showing how local analytics can guide budgeting. Crucially, this entire interaction—query, inference, and result—remained on the user’s device; no data was transmitted externally at any point.

9 Conclusion

We presented ExpenseSense, a human-centered evaluation of privacy-preserving, on-device financial tool calling using sub-2B quantized models. Our central finding is that local inference at this scale is not just technically feasible but practically reliable: Qwen3.5-0.8B in dual mode achieves 89.1% Task Accuracy with 3.0 s latency, keeping all user data—queries, context, and results—

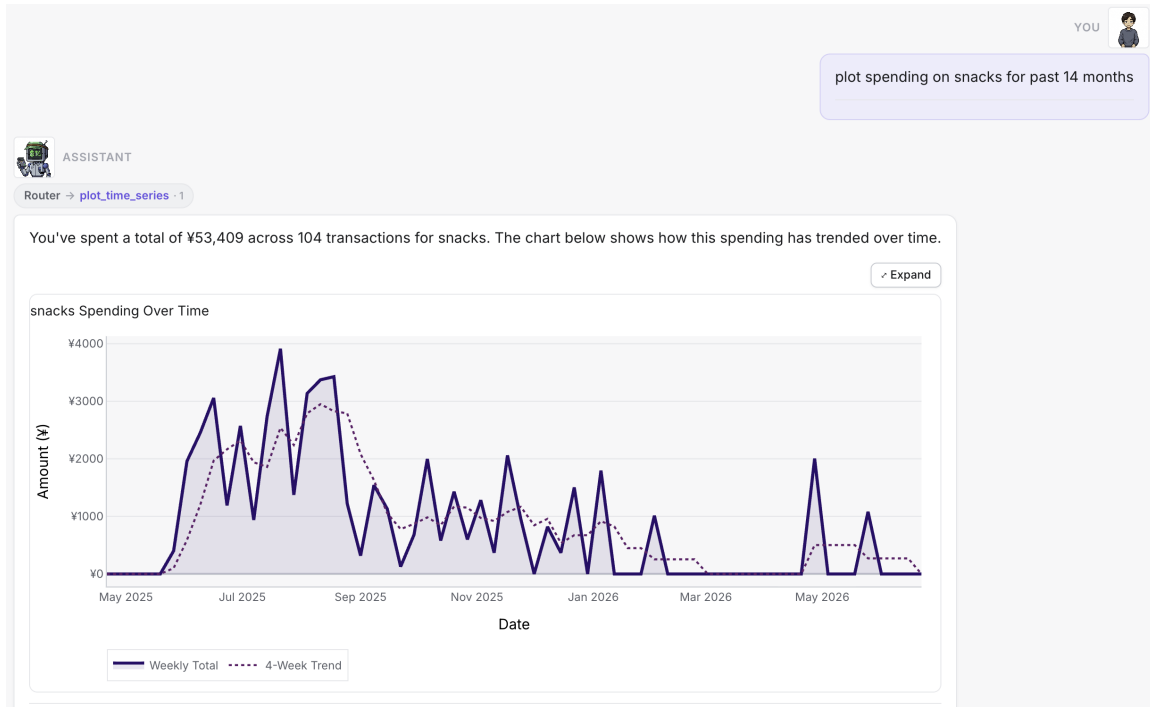


Figure 6: Real usage session: a user queries snack spending trends in informal language. The dual-agent Qwen3.5-0.8B pipeline correctly invokes `plot_time_series`, producing an actionable spending trend—entirely on-device with no data transmitted externally.

strictly on-device. Architectural decomposition into intent routing and parameter extraction further improves strict correctness (from 60.2% to 77.6% Correct Ratio) and, critically, shifts error modes toward safer failures: spurious parameter additions, which can silently corrupt financial analytics and mislead user decisions, are replaced by missing-parameter errors that are more visible and less harmful.

Several directions warrant future investigation. First, *fine-tuning* the best-performing models on domain-specific function-calling data could address the residual L3 failures while preserving on-device deployment. Second, extending the benchmark to *multi-turn* interactions would capture realistic conversational patterns, including follow-up queries where prior context affects parameter inference. Third, *user studies* examining how people understand and verify on-device privacy guarantees would complement our architectural analysis with evidence about user trust and mental models of local inference. Fourth, *multilingual* query support would broaden applicability across diverse user populations. Finally, evaluating lower-bit quantisation formats (Q4_K_M) could further reduce the memory footprint, making privacy-preserving financial AI accessible on lower capacity devices

such as smartphones.

Limitations

- All models use Q8_0 quantisation; lower-bit formats (Q4_K_M) may yield different accuracy–efficiency tradeoffs.
- Queries are in English and reflect a Japan-based personal finance context. Multilingual and cross-cultural diversity is not covered.
- Ground-truth labels are deterministic; some queries may admit legitimate alternative tool selections.
- Test cases were authored by a single developer; crowdsourcing or user study collection would improve query diversity.
- We do not evaluate the *downstream correctness* of the financial analytics produced only whether the tool call is correctly formed. Production deployment would require additional safeguards against calculation errors.
- Our privacy analysis is architectural (no data leaves the device) rather than cryptographic; we do not address model inversion or side-channel attacks on the local device itself.

Ethics Statement

ExpenseSense is designed for on-device inference specifically to preserve user financial privacy. No real user financial data was used in the benchmark; all test cases reference synthetic expense categories. The system is intended to complement, not replace, professional financial advice. We recognise that even on-device models may produce incorrect financial summaries, and advocate for clear user-facing disclaimers in production deployments.

Data and Code Availability

Code, prompts, benchmark data, and full results will be released on GitHub upon acceptance. Released artefacts include: all 115 test cases with ground-truth labels, the complete router and specialist prompt templates, inference configuration details, raw and validated CSV result files, and the benchmark runner, plotting scripts, and the exact model GGUFs used in this study to ensure complete reproducibility.

A Complexity Scoring Features

Feature	Range	Description
n_p	0+	Number of expected parameters
d_{date}	0–4	Date complexity: none / year / month / range / specific day
d_{cat}	0–2	Category distance: exact / alias / semantic
d_{rel}	0–1	Relative time expression (“past X months”)
d_{multi}	0–2	Compound parameter groups
d_{abbr}	0–1	Abbreviated year formats (“’24”)
d_{hol}	0–2	Holiday/calendar knowledge required

Table 7: Seven features of the complexity scoring function (Eq. 1).

B Known Category Vocabulary

The post-hoc parameter validation layer (Section 4.3) and the LLM prompts rely on a known category vocabulary defined dynamically or in static schema files. Table 8 lists the 11 major categories defined in the application alongside the mapped subcategories/aliases from the system configuration (`categories.json`) that are passed in the `{metadata}` block.

Major Category	Mapped Minor Categories / Subcategories
Housing and Utilities	housing, internet bill, electricity bill, gas bill, water & sewage bill, phone bill
Food	grocery, snacks, cafe, café, coffee, bento, beverage, combini meal, dining, eating with friend
Transportation	commute, ride share, tokyo metro, Tokyo Metro, flight tickets, cable car, bus, shinkansen, car rental, taxi
Accommodation	stay
Fitness	supplements, shoes, sports event, sports watch, sports clothing, sports rental, gym, sports equipment, basketball game, football game, futsal game
Souvenirs/Gifts/Treats	souvenirs, treat, gift, souvenirs/gifts/treats
Household and Clothing	clothing, household
Entertainment	entertainment, nomikai, activities, Activities, arcades & karaoke, Arcades & Karaoke, events & venues, Events & Venues
Miscellaneous	medicines, personal care, misc, help, charity, donation, entrance fees, park entrance fees, healthcare
Education	tuition, books, exam fees, notebooks, textbook
Electronics and Furniture	furniture, electronics

Table 8: Known category vocabulary from `categories.json`.

C Sample Test Cases

D Qualitative Error Examples

Table 10 illustrates representative failures for the dominant error types in Qwen3.5-2B (dual mode).

ID	Query	Tool	Expected Params	Tier
TS01	Can ya show spending on food for past 6 months?	plot_time_series	category=Food, months=6	L2
DI06	show breakdown of expenses for june 21 2024	plot_distribution	year=2024, month=6, day=21	L2
CP24	compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026	plot_comparison_bars	category=grocery, y1=2024, sm1=11, ey1=2025, em1=4, y2=2025, sm2=11, ey2=2026, em2=4	L3
CT07	sum total spent on metro for july 2024?	calculate_total	category=tokyo metro, year=2024, month=7	L2
TP06	top 5 expenses past month? disregard rent	get_top_expenses	n=5, months=1, ignore_rent=true	L2

Table 9: Sample test cases. Note informal language, abbreviations, and semantic mappings (“metro”→“tokyo metro”, “groceries”→“grocery”).

Error Type	Query	Expected	Model Output
PARAM_MISSING	“Show spending trend for the last 3 months”	{months: 3}	{ } — months omitted in the relative-time case
PARAM_VALUE_WRONG	“compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026”	{em1: 4, em2: 4}	{em1: 11, em2: 11} — confuses start month for end month in cross-year range
PARAM_EXTRA	“How much did I spend on food?”	{category: Food}	{category: Food, months: 1} — spurious time filter restricts scope to last month, silently returning an incorrect total

Table 10: Representative failure cases for Qwen3.5-2B (dual mode). PARAM_EXTRA is the most dangerous failure mode in financial contexts: the spurious months=1 filter silently restricts “total food spending” to “last month’s food spending”, returning a plausible but incorrect figure that a user may act on without realising the scope was changed.